ProjExD_Group14

Public

forked from c0a25046bf/ProjExD_Group14

About

No description, website, or topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors

No contributors

Languages

Python 100.0%

Suggested workflows

Based on your tech stack

Django

Build and Test a Django Project

Configure

Pylint

Lint a Python application with pylint.

Configure

SLSA Generic generator

Generate SLSA3 provenance for your existing release workflows

Configure

More workflows

Dismiss suggestions

C0A25062/当たり判定 had recent pushes 50 minutes ago Compare & pull request

2 Branches 0 Tags Go to file T Go to file Add file Code

This branch is up to date with c0a25046bf/ProjExD_Group14:main. Contribute Sync fork

c0a25046bf README変更 7b78f6c · 2 hours ago

README.md	README変更	2 hours ago
koukaton_up.py	基本コード	2 hours ago

README

Koukaton Up

実行環境の必要条件

- python >= 3.10
- pygame >= 2.1

ゲームの概要

- 主人公キャラクターをどんどん上に登らせるゲーム
- 参考URL：Only Up! ：Jump King

ゲームの遊び方

- キーを使って床をどんどん登って上に行く
- 落ちたら床があるところまで落ちる

ゲームの実装

共通基本機能

- 矢印でジャンプの方向決めてスペースの押す長さでジャンプ力を決める
- 壁に当たると跳ね返る

分担追加機能

- 1_グラフィック（画像）変更[ゆら],2_当たり判定の厳密化（頭ぶつけ・すり抜け防止）[うりゅう],3_ステージギミック追加[そら], 4_音楽[こうへい],5_左上のUI（高さ・情報の表示）[こな]

ToDoこな

メモ

- クラス内の変数は，すべて，「get_変数名」という名前のメソッドを介してアクセスするように設計してある
- すべてのクラスに関係する関数は，クラスの外で定義してある
  os.chdir(os.path.dirname(os.path.abspath(**file**)))

ProjExD_Group14 / koukaton_up.py

c0a25062d3 機能追加：当たり判定 7887599 · 28 minutes ago

293 lines (234 loc) · 10.2 KB

```python
import pygame
import sys
import os
import random

# カレントディレクトリの固定
os.chdir(os.path.dirname(os.path.abspath(__file__)))

WIDTH, HEIGHT = 600, 800
FPS = 60
GRAVITY = 0.6

BLACK = (20, 20, 25)
BLUE = (100, 150, 255)
GREEN = (120, 120, 120)
RED = (255, 100, 100)
WHITE = (255, 255, 255)
GOLD = (255, 215, 0)  # ゴールブロック用の金色

# ========================================
# 1. データ構造（クラス）の定義
# ========================================

class Player:
    def __init__(self):
        self._image = pygame.Surface((30, 40))
        self._image.fill(BLUE)
        self._rect = self._image.get_rect(center=(WIDTH // 2, HEIGHT - 100))

        self._vel_x = 0
        self._vel_y = 0
        self._on_ground = False

        self._is_charging = False
        self._charge_power = 0
        self._max_charge = 20.0
        self._direction = 1

        self._is_clear = False  # ゴールしたかどうかのフラグ

    # --- ゲッター ---
    def get_image(self): return self._image
    def get_rect(self): return self._rect
```

```python
        def get_vel_x(self): return self._vel_x
        def get_vel_y(self): return self._vel_y
        def get_on_ground(self): return self._on_ground
        def get_is_charging(self): return self._is_charging
        def get_charge_power(self): return self._charge_power
        def get_max_charge(self): return self._max_charge
        def get_direction(self): return self._direction
        def get_is_clear(self): return self._is_clear

        # --- セッター ---
        def set_vel_x(self, value): self._vel_x = value
        def set_vel_y(self, value): self._vel_y = value
        def set_on_ground(self, value): self._on_ground = value
        def set_is_charging(self, value): self._is_charging = value
        def set_charge_power(self, value): self._charge_power = value
        def set_direction(self, value): self._direction = value
        def set_is_clear(self, value): self._is_clear = value


class Platform:
    # 足場ごとに色を変えられるように引数追加
    def __init__(self, x, y, w, h, color):
        self._image = pygame.Surface((w, h))
        self._image.fill(color)
        self._rect = self._image.get_rect(topleft=(x, y))

    def get_image(self): return self._image
    def get_rect(self): return self._rect


# ========================================
# 2. クラスを操作する関数
# ========================================

def update_player(player, keys, platforms, goal_block):
    """プレイヤーの物理挙動と状態を更新する関数"""

    # ゴール済みの場合は入力を受け付けない（重力で落ちるだけ）
    if not player.get_is_clear():
        # 1. 地面にいる時の処理
        if player.get_on_ground():
            player.set_vel_x(0)

            if keys[pygame.K_LEFT] or keys[pygame.K_a]:
                player.set_direction(-1)
            if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
                player.set_direction(1)

            if keys[pygame.K_SPACE]:
                player.set_is_charging(True)
                current_power = player.get_charge_power()
                if current_power < player.get_max_charge():
                    player.set_charge_power(current_power + 0.4)
            else:
                if player.get_is_charging():
                    power = player.get_charge_power()
                    direction = player.get_direction()

                    player.set_vel_y(-power * 0.9 - 5)
```

```
                    player.set_vel_x(direction * (power * 0.4 + 2))

                    player.set_is_charging(False)
                    player.set_charge_power(0)
                    player.set_on_ground(False)

        # 2. 重力の適用
        if not player.get_on_ground():
            player.set_vel_y(player.get_vel_y() + GRAVITY)

        rect = player.get_rect()

        # X軸方向の移動と、足場の【左右】の当たり判定
        rect.x += player.get_vel_x()
        if rect.left < 0:
            rect.left = 0
            player.set_vel_x(player.get_vel_x() * -0.5)
            player.set_direction(1)
        if rect.right > WIDTH:
            rect.right = WIDTH
            player.set_vel_x(player.get_vel_x() * -0.5)
            player.set_direction(-1)

        # 追加：足場の左右側面との衝突をチェック
        for plat in platforms:
            plat_rect = plat.get_rect()
            if rect.colliderect(plat_rect):
                if player.get_vel_x() > 0:  # 右移動中に衝突
                    rect.right = plat_rect.left
                    player.set_vel_x(player.get_vel_x() * -0.5)
                    player.set_direction(-1)
                elif player.get_vel_x() < 0:  # 左移動中に衝突
                    rect.left = plat_rect.right
                    player.set_vel_x(player.get_vel_x() * -0.5)
                    player.set_direction(1)

        # Y軸方向の移動
        rect.y += int(player.get_vel_y())

        # --- 3. ゴール（天井）との当たり判定 ---
        goal_rect = goal_block.get_rect()
        if rect.colliderect(goal_rect):
            # 上昇中に天井の底にぶつかったらゴール
            if player.get_vel_y() < 0:
                rect.top = goal_rect.bottom
                player.set_vel_y(0)
                player.set_is_clear(True)

        # --- 4. 足場との当たり判定 ---
        player.set_on_ground(False)

        for plat in platforms:
            plat_rect = plat.get_rect()
            if rect.colliderect(plat_rect):
                # もともとの上面判定（落下中の着地判定）
                if player.get_vel_y() > 0:
                    if rect.bottom <= plat_rect.top + player.get_vel_y() + 1:
                        rect.bottom = plat_rect.top
                        player.set_vel_y(0)
```

```
                    player.set_vel_x(0)
                    player.set_on_ground(True)
                    break
            # 追加：上昇中に足場の【下側（天井部分）】に頭をぶつけた判定
            elif player.get_vel_y() < 0:
                rect.top = plat_rect.bottom
                player.set_vel_y(0)  # 上昇を止める
                break


# ========================================
# 3. メインループ
# ========================================

def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Jump King - 10 Floors to Goal")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont(None, 36)
    large_font = pygame.font.SysFont(None, 72)

    player = Player()

    # --- 10層分のマップ生成 ---
    platforms = []
    platforms.append(Platform(0, HEIGHT - 40, WIDTH, 40, GREEN)) # スタート床

    TOTAL_FLOORS = 10
    # ゴール（天井）のY座標：スタート床から 10層分(800px * 10) 上へ
    goal_y = (HEIGHT - 40) - (HEIGHT * TOTAL_FLOORS)

    # 金色の天井ブロックを生成
    goal_block = Platform(0, goal_y, WIDTH, 40, GOLD)

    # ゴールに到達するまで足場を生成し続ける
    platform_y = HEIGHT - 150
    while platform_y > goal_y + 100:
        w = random.randint(60, 130)
        x = random.randint(0, WIDTH - w)
        platforms.append(Platform(x, platform_y, w, 20, GREEN))
        gap_y = random.randint(80, 130)
        platform_y -= gap_y

    camera_y = 0
    max_height = 0

    running = True
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False

        keys = pygame.key.get_pressed()

        # 関数呼び出し（ゴールブロックも渡して判定させる）
        update_player(player, keys, platforms, goal_block)

        p_rect = player.get_rect()
```

```python

        # --- カメラ処理 ---
        SCROLL_TOP_MARGIN = 200
        if p_rect.y - camera_y < SCROLL_TOP_MARGIN:
            camera_y = p_rect.y - SCROLL_TOP_MARGIN

        SCROLL_BOTTOM_MARGIN = 150
        camera_easing_down = 0.08
        if p_rect.bottom - camera_y > HEIGHT - SCROLL_BOTTOM_MARGIN:
            target_camera_y = p_rect.bottom - (HEIGHT - SCROLL_BOTTOM_MARGIN)
            camera_y += (target_camera_y - camera_y) * camera_easing_down

        if camera_y > 0:
            camera_y = 0

        # --- スコア・階層計算 ---
        current_height = (HEIGHT - 40) - p_rect.bottom
        if current_height > max_height:
            max_height = current_height

        # 今いる階層（800px を 1階 とする）
        current_floor = min((current_height // HEIGHT) + 1, TOTAL_FLOORS)

        # --- 描画処理 ---
        screen.fill(BLACK)

        # ゴールの描画
        goal_rect = goal_block.get_rect()
        goal_draw_y = goal_rect.y - camera_y
        if -100 < goal_draw_y < HEIGHT + 100:
            screen.blit(goal_block.get_image(), (goal_rect.x, goal_draw_y))

        # 足場の描画
        for plat in platforms:
            plat_rect = plat.get_rect()
            draw_y = plat_rect.y - camera_y
            if -50 < draw_y < HEIGHT + 50:
                screen.blit(plat.get_image(), (plat_rect.x, draw_y))

        # プレイヤーの描画
        player_draw_y = p_rect.y - camera_y
        screen.blit(player.get_image(), (p_rect.x, player_draw_y))

        # チャージゲージと方向指示器
        if player.get_is_charging() and not player.get_is_clear():
            gauge_width = (player.get_charge_power() / player.get_max_charge()) * 40
            pygame.draw.rect(screen, RED, (p_rect.centerx - 20, player_draw_y - 15, gauge_width, 8))

        arrow_x = p_rect.right + 5 if player.get_direction() == 1 else p_rect.left - 10
        pygame.draw.rect(screen, WHITE, (arrow_x, player_draw_y + 15, 5, 5))

        # 情報表示（UI）
        ui_text = font.render(f"Floor: {current_floor} / {TOTAL_FLOORS}   Max: {max_height // 10}m", True, WHI
        screen.blit(ui_text, (10, 10))

        # ゴールした時の演出
        if player.get_is_clear():
            clear_text = large_font.render("CLEAR!!", True, GOLD)
            text_rect = clear_text.get_rect(center=(WIDTH // 2, HEIGHT // 2 - 50))
```

```
280                 screen.blit(clear_text, text_rect)
281
282                 sub_text = font.render("CONGRATULATIONS!", True, WHITE)
283                 sub_rect = sub_text.get_rect(center=(WIDTH // 2, HEIGHT // 2 + 10))
284                 screen.blit(sub_text, sub_rect)
285
286             pygame.display.flip()
287             clock.tick(FPS)
288
289         pygame.quit()
290         sys.exit()
291
292     if __name__ == "__main__":
293         main()
```

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also compare across forks or learn more about diff comparisons.

base: main ← compare: C0A25062/当たり判定

✓ **Able to merge.** These branches can be automatically merged.

Discuss and review the changes in this comparison with others. Learn about pull requests

Create pull request

1 commit | 1 file changed | 1 contributor

Commits on May 19, 2026

機能追加：当たり判定

c0a25062d3 committed 29 minutes ago

7887599

Showing 1 changed file with 26 additions and 6 deletions.

Split Unified

32 koukaton_up.py

```
112 112
113 113       rect = player.get_rect()
114 114
115     -     # 座標の更新（x軸）と壁の跳ね返り
    115 +     # x軸方向の移動と、足場の【左右】の当たり判定
116 116       rect.x += player.get_vel_x()
117 117       if rect.left < 0:
118 118           rect.left = 0
123 123           player.set_vel_x(player.get_vel_x() * -0.5)
124 124           player.set_direction(-1)
125 125
126     -     # 座標の更新（y軸）
    126 +     # 追加：足場の左右側面との衝突をチェック
    127 +     for plat in platforms:
    128 +         plat_rect = plat.get_rect()
    129 +         if rect.colliderect(plat_rect):
    130 +             if player.get_vel_x() > 0:  # 右移動中に衝突
    131 +                 rect.right = plat_rect.left
    132 +                 player.set_vel_x(player.get_vel_x() * -0.5)
    133 +                 player.set_direction(-1)
    134 +             elif player.get_vel_x() < 0:  # 左移動中に衝突
    135 +                 rect.left = plat_rect.right
    136 +                 player.set_vel_x(player.get_vel_x() * -0.5)
    137 +                 player.set_direction(1)
    138 +
```

	139	+	`    # Y軸方向の移動`
127	140		`    rect.y += int(player.get_vel_y())`
128	141		
129	142		`    # --- 3. ゴール（天井）との当たり判定 ---`
137	150		
138	151		`    # --- 4. 足場との当たり判定 ---`
139	152		`    player.set_on_ground(False)`
140		-	`    if player.get_vel_y() > 0:`
141		-	`        for plat in platforms:`
142		-	`            plat_rect = plat.get_rect()`
143		-	`            if rect.colliderect(plat_rect):`
	153	+	
	154	+	`    for plat in platforms:`
	155	+	`        plat_rect = plat.get_rect()`
	156	+	`        if rect.colliderect(plat_rect):`
	157	+	`            # もともとの上面判定（落下中の着地判定）`
	158	+	`            if player.get_vel_y() > 0:`
144	159		`                if rect.bottom <= plat_rect.top + player.get_vel_y() + 1:`
145	160		`                    rect.bottom = plat_rect.top`
146	161		`                    player.set_vel_y(0)`
147	162		`                    player.set_vel_x(0)`
148	163		`                    player.set_on_ground(True)`
149	164		`                    break`
	165	+	`            # 追加：上昇中に足場の【下側（天井部分）】に頭をぶつけた判定`
	166	+	`            elif player.get_vel_y() < 0:`
	167	+	`                rect.top = plat_rect.bottom`
	168	+	`                player.set_vel_y(0)  # 上昇を止める`
	169	+	`                break`
150	170		
151	171		
152	172		`# ========================================`